

Lab manual

Lab 1a - Set up Git

Setting up Git for PowerShell

Installing and configuring git - <https://git-scm.com/downloads>

- `git config --global user.name "FirstName LastName"`
- `git config --global user.email "githubusername@users.noreply.github.com"`
- `git config --global init.defaultBranch main`
- `git config --global credential.helper store`

Install posh-git -

<https://github.com/dahlbyk/posh-git>

- Install from the PowerShell Gallery
- Add to profile - automatically get new prompt

Downloads

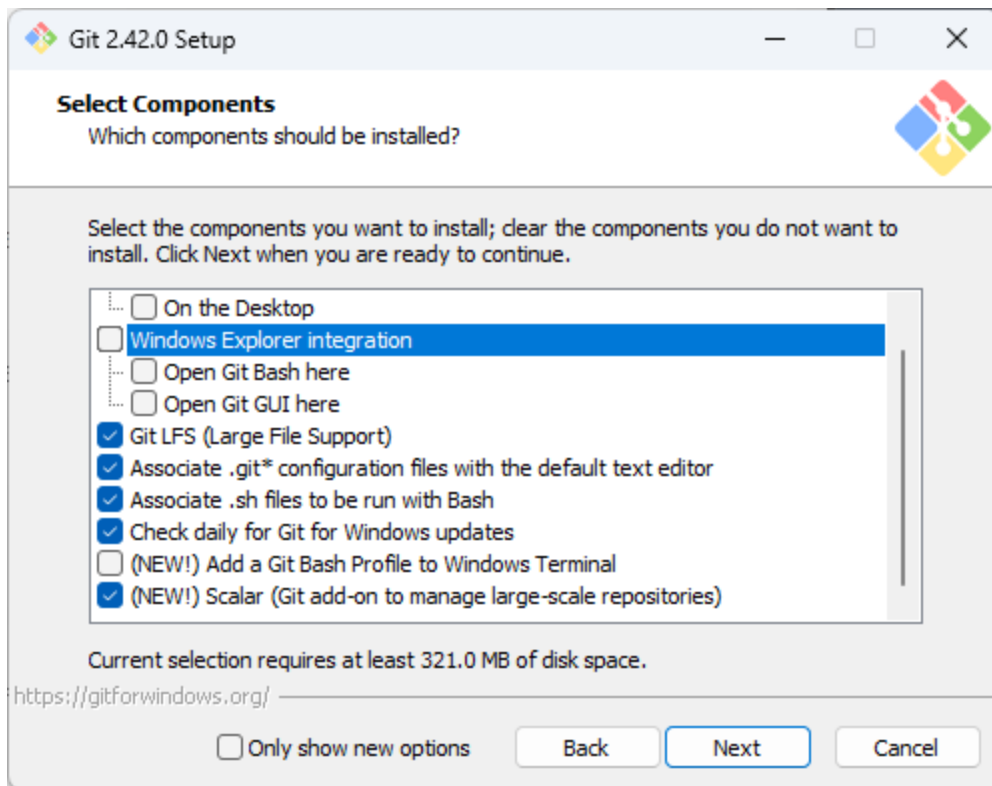


The goal of this lab is to install and configure Git and posh-git for PowerShell.

Step 1 - Install Git for Windows

Download the latest version of Git for the platform you are using. There are versions for Windows, macOS, and Linux. <https://git-scm.com/downloads>

The installer for Windows presents a series of pages with configuration options. The following lists show the recommended settings for each page of the installer.



Select Components

- Additional icons - optional (my preference = don't install)
- Windows Explorer integration - optional (my preference = don't install)
- Git LFS (Large File Support) - depends on your project needs (my preference = install)
- Associate .git* configuration files with the default text editor
- Associate .sh files to be run with Bash - if you are using Git bash
- Use VS Code as Git's default editor
- Check daily for Git for Windows updates - optional (my preference = install)
- (NEW!) Add a Git Bash Profile to Windows Terminal - optional (your preference)
- (NEW!) Scalar (Git add-on to manage large-scale repositories) - recommended

Choosing the default editor used by Git

- Use VS Code as Git's default editor

Adjusting the name of the initial branch in new repositories

- Override the default branch name for new repositories
 - Name = main

Adjusting your PATH environment

- Git from the command line and also from 3rd-party software

Choosing the SSH executable

- Use bundled OpenSSH - default

- Use external OpenSSH - if you have installed a different version of OpenSSH

Choosing HTTPS transport backend

- Use the native Windows Secure Channel library

Configuring the line edit conversions

- Checkout Windows-style, commit Unix-style line endings

Configuring the terminal emulator to use with Git Bash

- Use Windows' default console window

Choose the default behavior of `git pull`

- Default (Fast-forward or merge)

Choose a credential helper

- Git Credential Manager

Configuring extra options

- Enable file system caching
- Enable symbolic links

Configuring experimental options

- Enable experimental support for pseudo consoles
- Enable experimental built-in file system monitor

Step 2 - Post installation configuration

Before working with a repository you need to configure Git with your name and email address. You can also set the default branch name and configure Git to store your credentials. You may also want to adjust the colors used by Git depending on the terminal settings you are using.

```
git config --global user.name "FirstName LastName"
git config --global user.email "githubusername@users.noreply.github.com"
git config --global color.ui true
git config --global color.status.changed "magenta bold"
git config --global color.status.untracked "red bold"
git config --global color.status.added "red bold"
git config --global color.unmerged "yellow bold"
git config --global color.branch.remote "magenta bold"
git config --global color.branch.upstream "blue bold"
git config --global color.branch.current "green bold"
git config --global core.excludesfile ~/.gitignore
```

For more information, see the [Customizing Git](#) topic in the Git documentation.

When you are installing on Windows, the installer walks you through all the settings. If you are installing on macOS or Linux, you need to configure the default branch name and credential manager.

```
git config --global init.defaultBranch main
git config --global credential.helper store
```

Step 3 - Git ignore settings

Creating the .gitignore as a global configuration ensures that you are ignoring the same files across all repositories.

The tools you use to create and edit your content may create hidden, system, or temporary files that you do not want Git to sync to Github. Also, you can create workspace-specific settings for VS Code in a `.vscode` folder in your repository. This folder can contain code snippets or style sheets that you use in VS Code for that group of content. A `.gitignore` file tells Git which files and folders ignore for change tracking. You can create a global file in `$HOME\.gitignore`.

```
.vscode/

# Windows image file caches
Thumbs.db
ehthumbs.db

# Folder config file
Desktop.ini

# Recycle Bin used on file shares
$RECYCLE.BIN/

# Windows Installer files
*.cab
*.msi
*.msm
*.msp

# Windows shortcuts
*.lnk

# macOS files
.DS_Store
.AppleDouble
.LSOverride
```

The goal of this lab is to install posh-git and understand how to use it.

Step 1 - Install posh-git

posh-git is a PowerShell module that integrates Git with PowerShell. It provides Git status summary information that's displayed in a customized PowerShell prompt. **posh-git** also provides tab completion support for git commands, branch names, paths, and more.

For more information, see:

- [posh-git on the PowerShell Gallery](#)
- [posh-git on GitHub](#)

Install **posh-git** using the following command:

```
Install-Module posh-git
```

Step 2 - Use posh-git

Add the following command to your profile script.

```
Import-Module posh-git
```

Once **posh-git** is installed and imported, you'll see Git status summary information in your prompt.

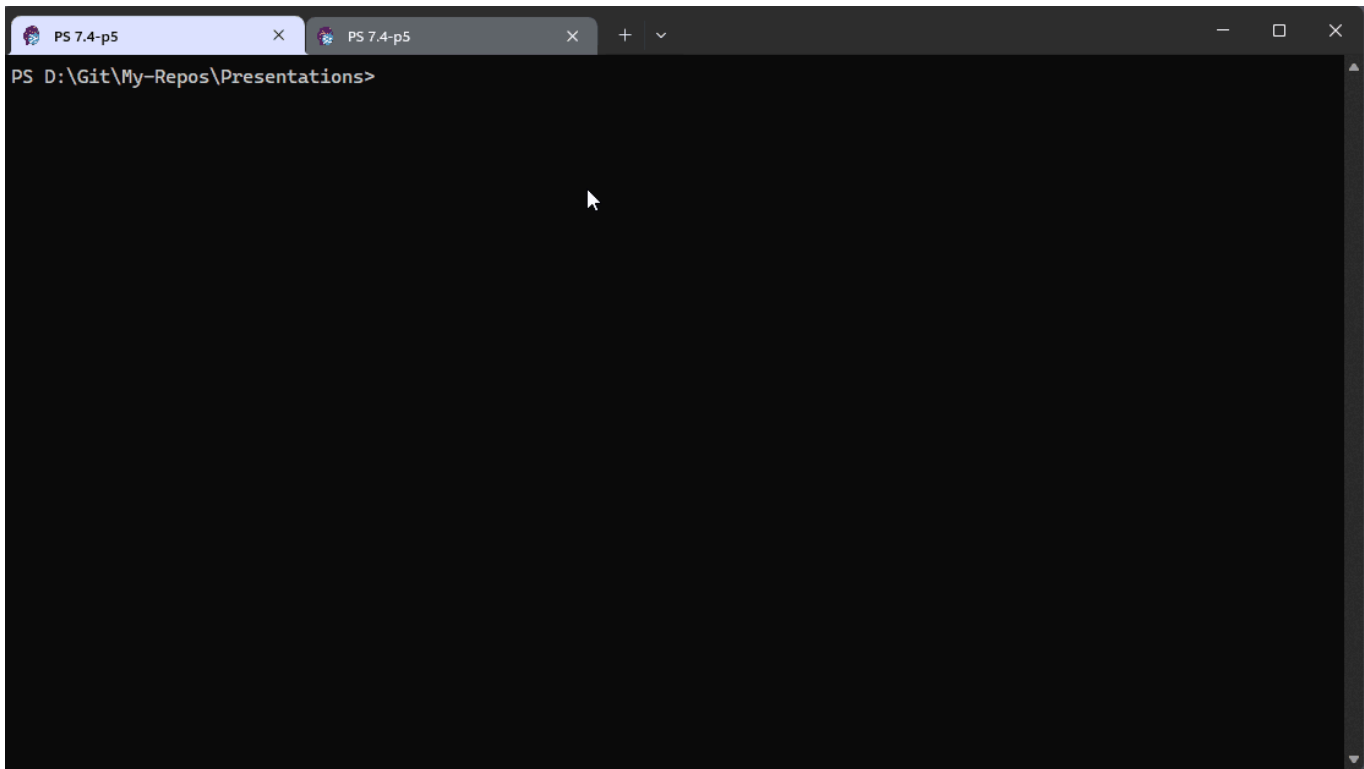
You should add the **Import-Module** command to your PowerShell profile script. If you haven't created a profile script yet, you can create one using the following command:

```
New-Item -Path $PROFILE -ItemType File -Force
```

Then, open the profile script in a text editor and add the command.

Step 3 - Explore the posh-git prompt

The Git status summary information provides a wealth of "Git status" information at a glance, all the time in your prompt.



By default, the status summary has the following format:

```
[{HEAD-name} S +A ~B -C !D | +E ~F -G !H W]
```

- [(BeforeStatus)
- {HEAD-name} is the current branch, or the SHA of a detached HEAD
 - Cyan means the branch matches its remote
 - Green means the branch is ahead of its remote (green light to push)
 - Red means the branch is behind its remote
 - Yellow means the branch is both ahead of and behind its remote
- S represents the branch status in relation to the remote (tracked origin) branch.

Note: This status information reflects the state of the remote tracked branch after the last `git fetch/pull` of the remote. Execute `git fetch` to update to the latest on the default remote repo. If you have multiple remotes, execute `git fetch --all`.

- == The local branch is at the same commit level as the remote branch (BranchIdenticalStatus)
- ↑<num> = The local branch is ahead of the remote branch by the specified number of commits; a `git push` is required to update the remote branch (BranchAheadStatus)
- ↓<num> = The local branch is behind the remote branch by the specified number of commits; a `git pull` is required to update the local branch (BranchBehindStatus)

- `<a>±<b>` = The local branch is both ahead of the remote branch by the specified number of commits (a) and behind by the specified number of commits (b); a rebase of the local branch is required before pushing local changes to the remote branch (`BranchBehindAndAheadStatus`). NOTE: this status is only available if `$GitPromptSettings.BranchBehindAndAheadDisplay` is set to `Compact`.
- `x` = The local branch is tracking a branch that is gone from the remote (`BranchGoneStatus`)
- `ABCD` represent the index; `|` (`DelimStatus`); `EFGH` represent the working directory
 - `+` = Added files
 - `~` = Modified files
 - `-` = Removed files
 - `!` = Conflicted files
 - As with `git status` output, index status is displayed in dark green and working directory status in dark red
- `W` represents the overall status of the working directory
 - `!` = There are unstaged changes in the working tree (`LocalWorkingStatusSymbol`)
 - `~` = There are uncommitted changes i.e. staged changes in the working tree waiting to be committed (`LocalStagedStatusSymbol`)
 - None = There are no unstaged or uncommitted changes to the working tree (`LocalDefaultStatusSymbol`)
- `]` (`AfterStatus`)

The symbols and surrounding text can be customized by the corresponding properties on `$GitPromptSettings`.

For example, a status of `[main ≡ +0 ~2 -1 | +1 ~1 -0]` corresponds to the following `git status`:

```
# On branch main
#
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
#       modified:   this-changed.txt
#       modified:   this-too.txt
#       deleted:    gone.ps1
#
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   not-staged.ps1
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
```

```
#  
# new.file
```

Lab 2 - Create a new repository and add files

The goal of this lab is to create a new repository with basic content and push it to GitHub.

Prerequisites:

- A GitHub account
- Tools installed on your local machine as configured in Lab 1:
 - Visual Studio Code
 - Git with configuration
 - PowerShell with the **posh-git** module

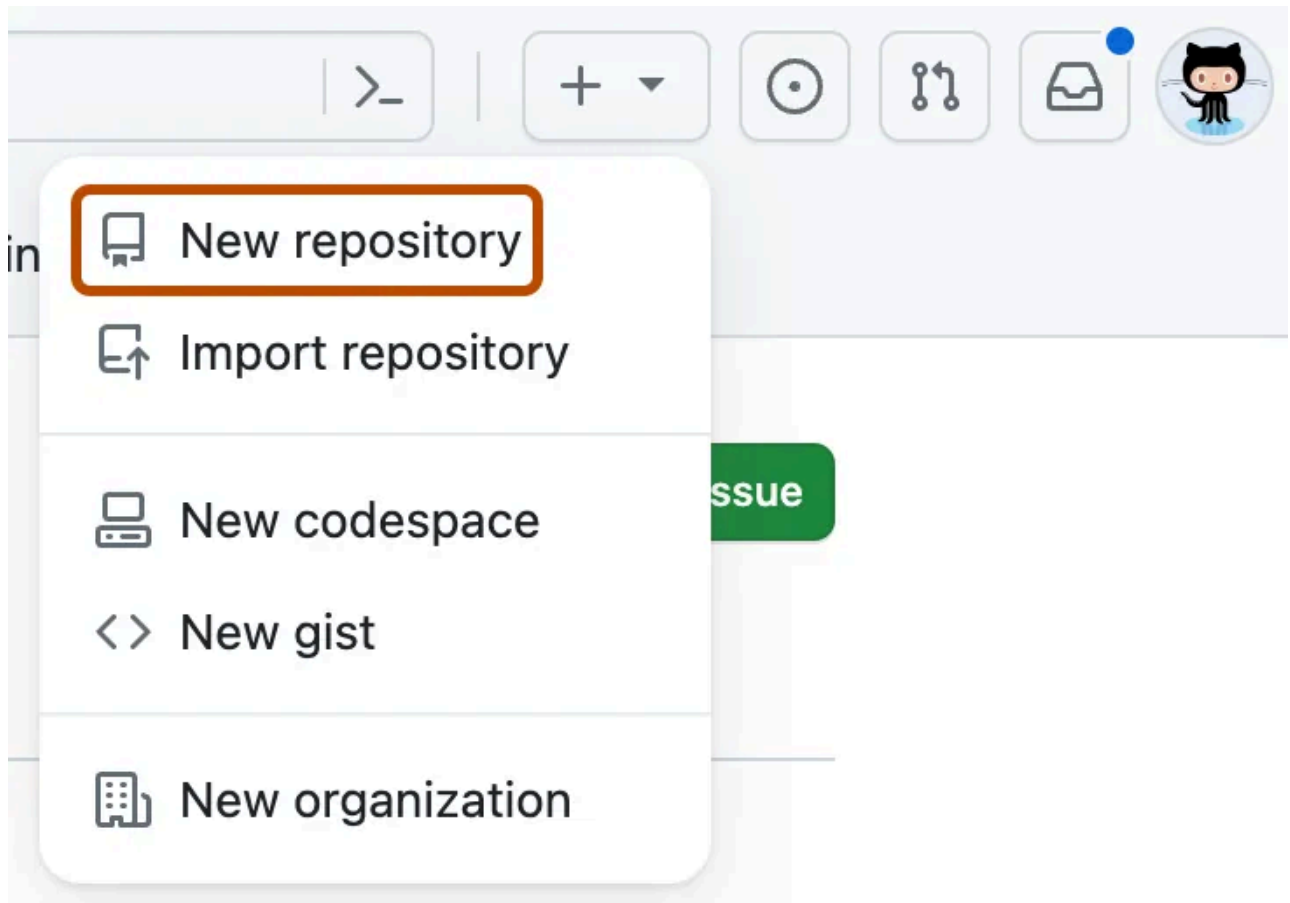
In this lab you will:

1. Create a new repository on GitHub and then clone it to your local machine.
2. Create a new branch in your local repository, add and edit files, then push your branch to your GitHub repository.
3. Merge your branch back into the main branch using a pull request in GitHub.
4. Sync the main branch to your local repository and clean up the working branches.

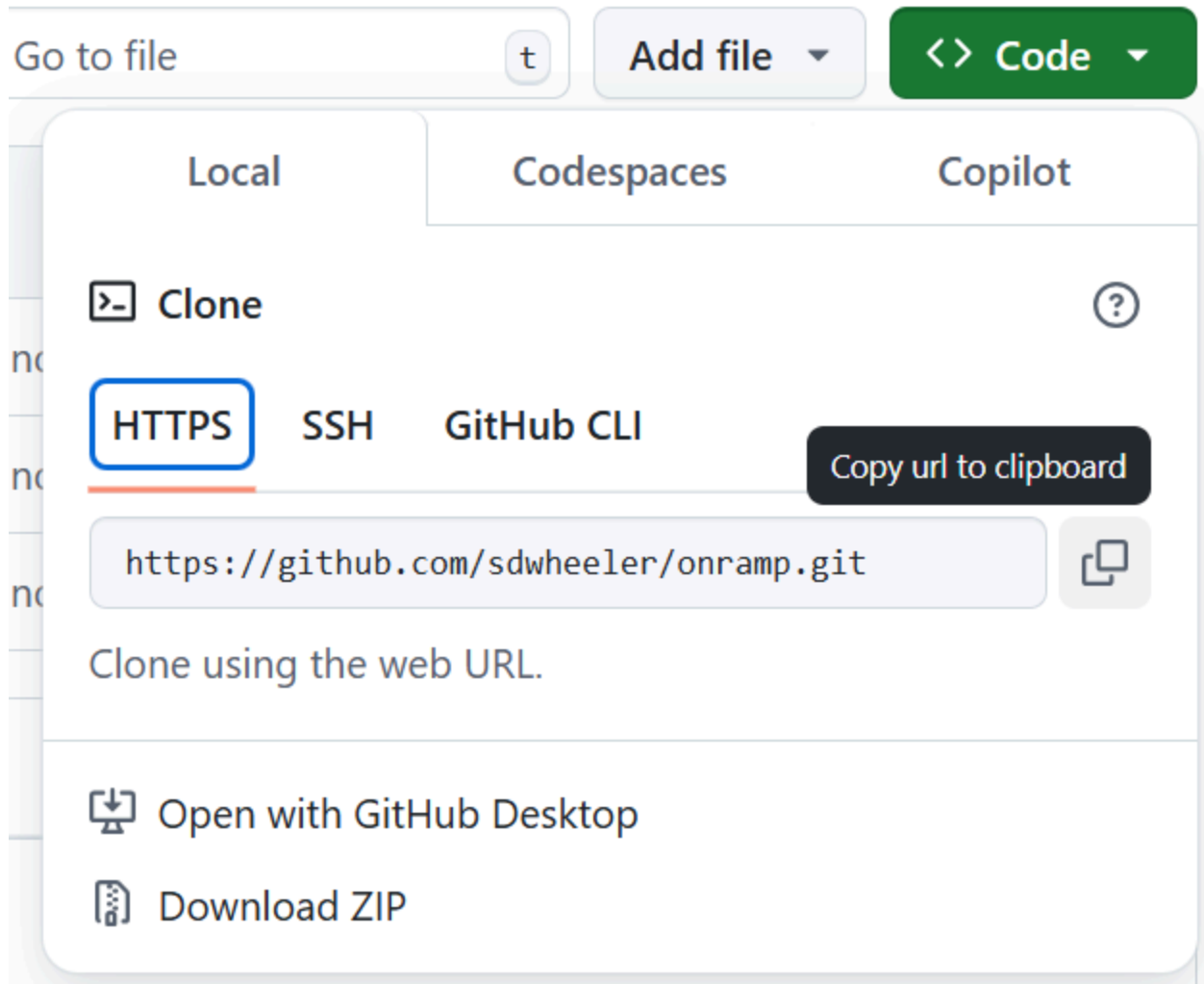
Step 1 - Create a new repository on GitHub and clone it

GitHub lets you add a README file at the same time you create your new repository. GitHub also offers other common options such as a license file, but you don't have to select any of them now.

1. In the upper-right corner of any page, select **+**, then select **New repository**.



2. In the **Repository name** box, type in a name for your project (no spaces or special characters). For this lab, choose a name that's different from your lab partner's name. For example, you could add your initials to create a name like `sdwGitFun`.
3. In the **Description** box, type a short description. For example, type "This repository is for practicing the GitHub Flow."
4. You can choose to make your repository **Public** or **Private**. Select **Public**.
5. Select **Add a README** file.
6. Select **Create** repository.
7. Copy the URL of the repository. You need this URL to clone the repository to your local machine.



8. In your terminal, navigate to the directory where you want to clone the repository. For example, type `cd ~/Git` and press Enter.
9. Type `git clone <repository URL>` and press Enter.

Git creates a new directory with the same name as the repository and copies all of the files from the remote repository to your local machine.

Step 2 - Create a new branch, make local changes, and push to GitHub

Now take a few minutes to add files to the repository. You can add any files you want, but here are a few suggestions:

1. Create a new working branch using the following command:

```
git checkout -b add-meta-files
```

2. Create a new file called `LICENSE.md` and add a license to the file. You can use the [ChooseALicense website](#) to help you choose a license.

3. Create a new file called `CONTRIBUTING.md` and add a short description of how to contribute to the project.
4. Create a new file called `CHANGELOG.md` and add a short description of the changes made to the project.
5. Use the commands you learned in [Tracking changes][01] to add the files to the staging area and commit them.

```
git add -A
git commit -m 'Add license, contributing, and changelog files'
```

6. Push your changes to GitHub:

```
git push origin add-meta-files
```

Step 3 - Merge your branch in a PR and sync to local

1. Go to your repository on GitHub and select the **Pull requests** tab.
2. Select the **New pull request** button.
3. Make sure the base branch is `main` and the compare branch is your working branch (e.g. `add-meta-files`).
4. Select the **Create pull request** button.
5. Fill out the pull request description and select **Submit**.
6. Review the pull request and select **Merge pull request** to merge your changes into the main branch.

Step 4 - Sync your local repository and clean up working branches

1. Go back to your terminal and make sure you're on the main branch:

```
git checkout main
```

2. Pull the latest changes from GitHub to your local repository:

```
git pull origin main
```

This updates your local main branch with the changes you just merged in GitHub.

3. Delete your working branch if you no longer need it:

```
git push origin --delete add-meta-files
git branch -D add-meta-files
```

The `git push` command deletes the branch in your GitHub repository. The `git branch` command deletes the working branch and the tracking branch reference from your local repo.

Lab 3 - Partner workflow - Fork and clone

In this lab, you will pair up with another student to practice the GitHub fork and pull request workflow. This is the standard workflow for contributing to open source projects or any project where you don't have write access to the repository.

Prerequisites

- You should have completed Lab 2 and have a repository on GitHub with some content
- Exchange GitHub repo and usernames with your lab partner

Lab objectives

In this lab you will:

1. Fork your partner's repository
2. Clone it to your local machine
3. Add the upstream remote
4. Create a new branch
5. Add a new file or make changes to an existing file
6. Commit the changes
7. Push the changes to your fork
8. Submit a pull request to the original repository
9. Review and merge your partner's pull request

Step 1 - Fork your partner's repository

1. Browse to your partner's repository on GitHub: `https://github.com/<partner-username>/<init>GitFun`
2. Select the **Fork** button in the top right corner
3. In the **Choose an owner** dropdown, select your personal account
4. Select the **Create fork** button

GitHub creates a copy of your partner's repository in your account.

Step 2 - Clone your fork to your local machine

1. In your terminal, navigate to your Git directory: `cd ~/Git`
2. Clone your fork:

```
git clone https://github.com/<your-username>/<repository-name>.git
```

Alternatively, you can use the GitHub CLI:

```
gh repo clone <your-username>/<repository-name>
```

3. Change to the repository directory:

```
cd <repository-name>
```

Step 3 - Add the upstream remote

This step links your local repository to the original repository (your partner's repo). This allows you to pull changes from the original repository to keep your fork in sync. If you cloned the fork using the GitHub CLI, the upstream remote is added automatically. If you cloned using Git, you need to add the upstream remote manually:

```
git remote add upstream https://github.com/<partner-username>/<repository-name>.git
```

Verify the remotes:

```
git remote -v
```

You should see both **origin** (your fork) and **upstream** (your partner's original repo).

Step 4 - Create a working branch

Always create a new branch for your changes. Never work directly on the main branch.

```
git checkout -b add-my-contribution
```

Step 5 - Make changes

Using Visual Studio Code, make some changes to the repository. Here are some ideas:

- Add a new file called **contributors.md** and add your name
- Add a new section to the README
- Create a new file with some content related to the project
- Fix a typo or improve existing documentation

Step 6 - Stage and commit your changes

```
git add -A
git commit -m "Add my contribution to the project"
```

Step 7 - Push your working branch to your fork

```
git push origin add-my-contribution
```

Step 8 - Submit a pull request

1. Go to your partner's original repository on GitHub: <https://github.com/<partner-username>/<repository-name>/pulls>
2. Select the **New pull request** button
3. Select **compare across forks**
4. Set up the pull request:
 - **Base repository:** [<partner-username>/<repository-name>](#) **base:** `main`
 - **Head repository:** [<your-username>/<repository-name>](#) **compare:** `add-my-contribution`
5. Fill out the pull request description:
 - Add a title (e.g., "Add contributions from [your name]")
 - Add a description of what you changed and why
6. Select **Create pull request**

Step 9 - Review your partner's pull request

Now your partner should have submitted a pull request to YOUR repository. Let's review and merge it:

1. Go to your repository on GitHub
2. Select on the **Pull requests** tab
3. Select on your partner's pull request
4. Review the changes:
 - Look at the **Files changed** tab to see what was modified
 - Add comments if you have feedback
 - Select **Review changes** and select **Approve**
5. Select **Merge pull request**
6. Select **Confirm merge**
7. Optionally, delete the branch after merging

Step 10 - Sync your local repository

After the PR is merged, update your local repository:

```
git checkout main
git pull origin main
```

Step 11 - Cleanup your working branch

Delete the branch you used for your contribution:

```
git push origin --delete add-my-contribution  
git branch -D add-my-contribution
```

Bonus: Keep your fork in sync

As the original repository (upstream) receives changes, you'll want to keep your fork updated:

```
git checkout main  
git pull upstream main  
git push origin main
```